Design and Implementation of a Mini-RISC CPU Simulator

Department of Computer Science and Engineering

Objective

The objective of this project is to design and implement a software simulator of a simple RISC-style processor to understand:

- Fetch–Decode–Execute cycle
- Role of Program Counter (PC) and Instruction Register (IR)
- Interaction between registers, ALU, and control unit
- Execution of assembly programs step-by-step
- Generation of instruction execution trace

Processor Architecture

Registers

- 4 General Purpose Registers: R0, R1, R2, R3 (8-bit)
- Accumulator (ACC): 8-bit
- Program Counter (PC): 8-bit
- Instruction Register (IR): 8-bit

Flags

- Z: Zero flag
- C: Carry flag

Memory

- Instruction Memory: 256 bytes

ALU Operations

- ADD, SUB, MUL, DIV, POW, CMP

Instruction Format

Each instruction is 8 bits:

```
[ OPCODE (4 bits) ][ OPERAND (4 bits) ]
```

Instruction Set Architecture

Opcode	Mnemonic	Operation
0x0	NOP	No operation
0x1	LOAD Rn	ACC ← Rn
0x2	STORE Rn	Rn ← ACC
0x3	MOVI k	ACC ← k
0x4	ADD Rn	ACC ← ACC + Rn
0x5	SUB Rn	ACC ← ACC - Rn
0x6	MUL Rn	ACC ← ACC × Rn
0x7	DIV Rn	ACC ← ACC / Rn
0x8	POW Rn	ACC ← ACC ^Rn
0x9	CMP Rn	Set flags based on ACC - Rn
0xA	JMP addr	PC ← addr
0xB	JZ addr	If Z=1, PC ← addr
0xC	JC addr	If C=1, PC ← addr
0xF	HALT	Stop execution

Execution Cycle

```
FETCH:    IR ← MEM[PC]
DECODE:   Decode instruction
EXECUTE:  Perform operation
UPDATE:   Update flags
PC ← PC + 1 (unless jump)
```

Execution Rules

- All operands must be in registers
- All operations must happen inside CPU
- All results must go through ACC
- PC and IR must be used explicitly

Execution Trace Log

A file `cpu_trace.txt` must be generated:

```
[TIME] PC=05 IR=0x61 INST=MUL R1 | R0=5 R1=3 R2=0 R3=0 | ACC=15 | Z=0 C=0
```

C Skeleton Code

```c
/*
============================================================
  Mini-RISC CPU Simulator
============================================================
*/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MEM_SIZE 256

typedef struct {
    unsigned char R[4];
    unsigned char ACC;
    unsigned char PC;
    unsigned char IR;
    unsigned char Z;
    unsigned char C;
} CPU;

unsigned char MEMORY[MEM_SIZE];
FILE *logfp = NULL;

void open_log() {
    logfp = fopen("cpu_trace.txt", "a");
    if (!logfp) {
        printf("Cannot open log file!\n");
        exit(1);
    }
}

void log_state(CPU *cpu, const char *instr) {
    time_t t = time(NULL);
    struct tm *tm = localtime(&t);

    fprintf(logfp,
        "[%02d:%02d:%02d] PC=%d IR=0x%02X INST=%s | "
        "R0=%d R1=%d R2=%d R3=%d | ACC=%d | Z=%d C=%d\n",
        tm->tm_hour, tm->tm_min, tm->tm_sec,
        cpu->PC, cpu->IR, instr,
        cpu->R[0], cpu->R[1], cpu->R[2], cpu->R[3],
        cpu->ACC, cpu->Z, cpu->C
    );
}

void reset_cpu(CPU *cpu) {
    memset(cpu, 0, sizeof(CPU));
```

```
}

void fetch(CPU *cpu) {
    cpu->IR = MEMORY[cpu->PC];
}

int execute(CPU *cpu) {
    unsigned char opcode = (cpu->IR >> 4) & 0x0F;
    unsigned char operand = cpu->IR & 0x0F;

    switch (opcode) {
        case 0x0: break; // NOP
        case 0x1: cpu->ACC = cpu->R[operand]; break; // LOAD
        case 0x2: cpu->R[operand] = cpu->ACC; break; // STORE
        case 0x3: cpu->ACC = operand; break; // MOVI
        // Remaining instructions to be implemented by students
        case 0xF: return 0; // HALT
        default: printf("Unknown opcode!\\n"); return 0;
    }

    cpu->PC++;
    return 1;
}

void load_program(const char *filename) {
    FILE *fp = fopen(filename, "r");
    if (!fp) {
        printf("Cannot open program file!\\n");
        exit(1);
    }

    int addr = 0;
    unsigned int instr;
    while (fscanf(fp, "%x", &instr) != EOF) {
        MEMORY[addr++] = (unsigned char)instr;
    }
    fclose(fp);
}

int main() {
    CPU cpu;
    reset_cpu(&cpu);
    open_log();

    load_program("program.hex");

    int running = 1;
    while (running) {
        fetch(&cpu);
        running = execute(&cpu);
        log_state(&cpu, "TODO");
```

```
    }

    fclose(logfp);
    return 0;
}
```

Submission

- Source code
- Sample assembly programs
- cpu_trace.txt
- 2–3 page report